



Il Livello Logico-Digitale

Reti combinatorie



Sommario

- ❑ Il segnale binario
- ❑ Algebra di Boole e funzioni logiche
- ❑ Porte logiche
- ❑ Analisi e sintesi di circuiti combinatori



Segnali e informazioni

- ❑ Per elaborare informazioni, occorre rappresentarle (o codificarle)
- ❑ Per rappresentare (o codificare) le informazioni si usano segnali
- ❑ I segnali devono essere elaborati, nei modi opportuni, tramite dispositivi di elaborazione
- ❑ In un **sistema digitale** le informazioni vengono **rappresentate**, **elaborate** e **trasmesse** mediante grandezze fisiche (segnali) che si considerano assumere solo **valori discreti**. Ogni valore è associato a una cifra (**digit**) della rappresentazione.



Il segnale binario

- **Segnale binario**: una grandezza che può assumere due valori distinti, convenzionalmente indicati con 0 e 1
 - $s \in \{0, 1\}$

- **Grandezze fisiche** utilizzate in un sistema digitale per la rappresentazione dell'informazione:
 - ⇒ segnali **elettrici** (**tensione**, corrente)
 - ⇒ grandezze di tipo magnetico (stato di magnetizzazione)
 - ⇒ segnali ottici



Algebra di Commutazione

- ❑ Deriva **dall'algebra di Boole** e consente di descrivere matematicamente i circuiti digitali (o circuiti logici)
- ❑ Definisce le **espressioni logiche** che descrivono il **comportamento** del circuito da realizzare nella forma $U = f(I)$
- ❑ A partire dalle equazioni logiche è possibile derivare la **realizzazione circuitale** (rete logica)
- ❑ I componenti dell'algebra di Boole sono: le variabili di commutazione, gli operatori fondamentali e le proprietà degli operatori logici tramite le quali è possibile trasformare le espressioni logiche



Variabili e operatori logici

- Una **variabile di commutazione** (o variabile logica) corrisponde al singolo bit di informazione rappresentata e elaborata
- Gli **operatori fondamentali** sono

Somma (OR)

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 1$$

Prodotto (AND)

$$0 \ 0 = 0$$

$$0 \ 1 = 0$$

$$1 \ 0 = 0$$

$$1 \ 1 = 1$$

Inversione (NOT)

$$!0 = 1$$

$$!1 = 0$$

Precedenza degli operatori: inversione, prodotto, somma



Proprietà degli operatori logici

Proprietà	AND ($\cdot$)	OR ($+$)
Identità	$1 \cdot A = A$	$0 + A = A$
Elemento nullo	$0 \cdot A = 0$	$1 + A = 1$
Idempotenza	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot !A = 0$	$A + !A = 1$
Commutativa	$A \cdot B = B \cdot A$	$A + B = B + A$
Associativa	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributiva	$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Assorbimento	$A \cdot (A + B) = A$	$A + A \cdot B = A$
De Morgan	$!(A \cdot B) = !A + !B$	$!(A + B) = !A \cdot !B$



Osservazioni

- ❑ Le proprietà commutativa, distributiva, identità e inverso sono postulati (cioè assunti veri per definizione).
- ❑ Le altre proprietà sono teoremi (dimostrabili per induzione matematica completa attribuendo alle variabili coinvolte sia il valore **0** che il valore **1**).



Principio di dualità

- Nell'algebra di Boole vale il principio di dualità.
- Il duale di una funzione booleana si ottiene sostituendo:
 - **and** a **or**,
 - **or** a **and**,
 - gli **0** agli **1**,
 - gli **1** agli **0**,
 - e sostituendo con (**!a**) la variabile (**a**).



Teorema di De Morgan

$$\neg(A \cdot B) = \neg A + \neg B$$

- La negazione dell'**and** di due variabili (cioè il **nand** di due variabili) equivale all'**or** delle variabili negate.

- Negando entrambi i membri delle due precedenti espressioni si ricava:

$$A \cdot B = \neg(\neg A + \neg B)$$

- L'**and** di due variabili equivale al **nor** delle variabili negate.



Teorema di De Morgan

$$\mathbf{!(A + B) = !A \cdot !B}$$

- La negazione dell'**or** di due variabili (cioè il **nor** di due variabili) equivale all'**and** delle variabili negate.
- Negando entrambi i membri delle due precedenti espressioni si ricava:

$$\mathbf{A + B = !(!A \cdot !B)}$$

- L'**or** di due variabili equivale al **nand** delle variabili negate.



Esempio di trasformazione

□ $F(a,b,c)=!a!bc + !abc + !ab!c$

Espressione trasformata	proprietà utilizzata
$!a!bc+!abc+!ab!c$	idempotenza $x+x=x$
$!a!bc+!abc+!abc+!ab!c$	distributiva $xy+xz=x(y+z)$
$!ac(!b+b) + !ab(c+!c)$	inverso $x+!x=1$
$!ac1 + !ab1$	identità $x1=x$
$!ac + !ab$	distributiva
$!a(c + b)$	



Insieme funzionalmente completo

- I tre operatori logici elementari (**and, or, not**) costituiscono un **insieme funzionalmente completo** cioè consentono di rappresentare una qualunque funzione di commutazione.
- **Problema:** Esistono insiemi di operatori più ridotti, che siano ancora un insieme funzionalmente completo?



Insieme funzionalmente completo

- Consideriamo l'insieme composto dai due operatori **(not, and)**: per vedere se è funzionalmente completo dobbiamo verificare se, mediante applicazioni anche ripetute di **not** e **and**, è possibile realizzare l'operatore **or**

- Dal teorema di De Morgan:

$$\neg(\neg A \cdot \neg B) = A + B$$

- Poichè è possibile realizzare l'operatore **or** componendo gli operatori **not** e **and** $\Rightarrow$ l'insieme **(not, and)** è funzionalmente completo.



Insieme funzionalmente completo

- Al contrario se consideriamo l'insieme composto dai due operatori (**and**, **or**) si dimostra che **non** è funzionalmente completo, perché la loro composizione non permette di ricavare in alcun modo l'operatore **not**.



Operatori universali

- ❑ **Problema:** L'insieme composto dal solo operatore **(nor)** è un insieme funzionalmente completo?
- ❑ Per vedere se l'operatore **nor** costituisce un insieme funzionalmente completo dobbiamo verificare se mediante il **nor** è possibile realizzare gli operatori **not**, **and**, e **or**:
 - $0 \text{ nor } A = !A$ oppure $(A \text{ nor } A) = !A$
 - $(A \text{ nor } B) \text{ nor } 0 = A \text{ or } B$
 - $(A \text{ nor } 0) \text{ nor } (B \text{ nor } 0) = A \text{ and } B$
- ❑ L'operatore **nor** è funzionalmente completo (**operatore universale**).



Operatori universali

- ❑ **Problema:** L'insieme composto dal solo operatore **(nand)** è un insieme funzionalmente completo?
 - ❑ Per vedere se l'operatore **nand** costituisce un insieme funzionalmente completo dobbiamo verificare se mediante il **nand** è possibile realizzare gli operatori **not**, **and**, e **or**.
 - $1 \text{ nand } A = !A$ oppure $(A \text{ nand } A) = !A$
 - $(A \text{ nand } B) \text{ nand } 1 = A \text{ and } B$
 - $(A \text{ nand } 1) \text{ nand } (B \text{ nand } 1) = A \text{ or } B$
 - ❑ Nota: Tali relazioni si possono ottenere da quelle viste per l'operatore **nor** applicando il principio di dualità.
 - ❑ L'operatore **nand** è funzionalmente completo (**operatore universale**).
-



Funzioni combinatorie

- ❑ Una **funzione combinatoria** (o funzione booleana, o funzione logica) corrisponde ad un'espressione booleana, contenente una o più variabili booleane e gli operatori booleani AND, OR e NOT
- ❑ Dando dei valori alle variabili booleane della funzione combinatoria, si calcola il corrispondente valore della funzione
- ❑ Esempio: funzione logica a 2 ingressi A e B e 2 uscite S e C
 - S=1 se e solo se uno degli ingressi vale 1, ma non entrambi
 - C=1 se e solo se entrambi gli ingressi valgono 1
- ❑ Espressioni booleane:
 - $S = A!B + !AB$
 - $C = AB$



Tabella di verità

- ❑ Per specificare il comportamento di una funzione combinatoria è possibile specificare, per ogni possibile configurazione degli ingressi, il valore dell'uscita tramite la **tabella di verità**
 - ❑ La tabella di verità di una funzione a n ingressi ha 2^n righe, che corrispondono a tutte le possibili configurazioni di ingresso: ad esempio per 2 ingressi la tabella ha 4 righe
 - ❑ La tabella di verità ha due gruppi di colonne:
 - **colonne degli ingressi**, le cui righe contengono tutte le combinazioni di valori delle variabili della funzione
 - **colonna dell'uscita**, che riporta i corrispondenti valori assunti dalla funzione
-



Esempio di Tabella di Verità

$n = 2$
ingressi

colonna
uscita

$2^n = 2^2 = 4$
righe

A	B	f = A and B
0	0	0
0	1	0
1	0	0
1	1	1



Esempio: Funzione di maggioranza

- Si consideri una funzione combinatoria dotata di 3 ingressi A, B e C, e di un'uscita F, funzionante come segue:
 - Se la maggioranza degli ingressi vale 0, l'uscita vale 0
 - Se la maggioranza degli ingressi vale 1, l'uscita vale 1



Tabella di verità della funzione di maggioranza

$n = 3$
ingressi

$2^n = 2^3 = 8$
righe

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

colonna
uscita



Porte logiche (circuiti combinatori elementari)

- ❑ I circuiti digitali sono formati da componenti digitali elementari, chiamati **porte logiche**
- ❑ Le porte logiche corrispondono agli **operatori elementari** dell'algebra di commutazione
- ❑ Le porte logiche vengono classificate in base al modo di funzionamento: porta **NOT**, porta **AND**, porta **OR** (sono le porte logiche fondamentali e costituiscono un insieme di operatori funzionalmente completo)
 - Classificazione per numero di ingressi: porte a 1 ingresso, porte a 2 ingressi, porte a 3 ingressi, e così via ...



Porta NOT (invertitore, negatore)

Simbolo funzionale

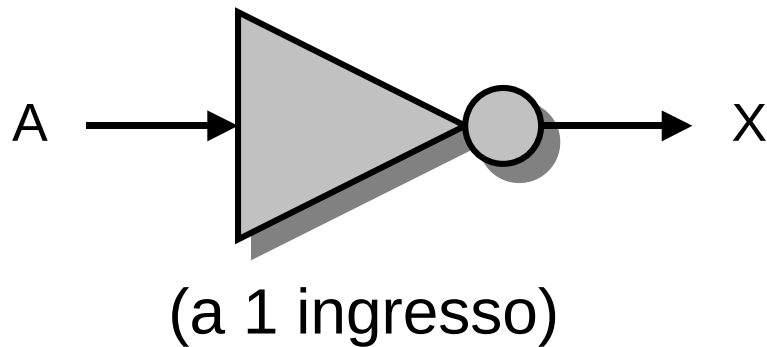
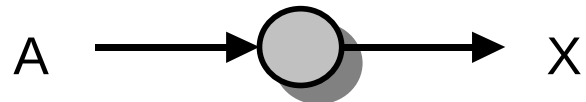


Tabella di verità

A	X
0	1
1	0



simbolo semplificato



Porta AND

Simbolo funzionale

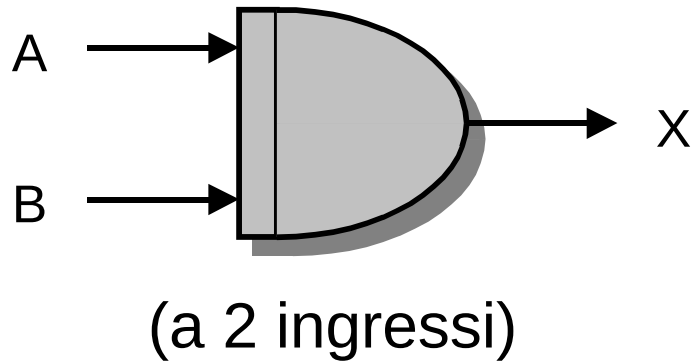


Tabella di verità

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

L'uscita vale 1 se e solo se entrambi gli ingressi valgono 1



Porta OR

Simbolo funzionale

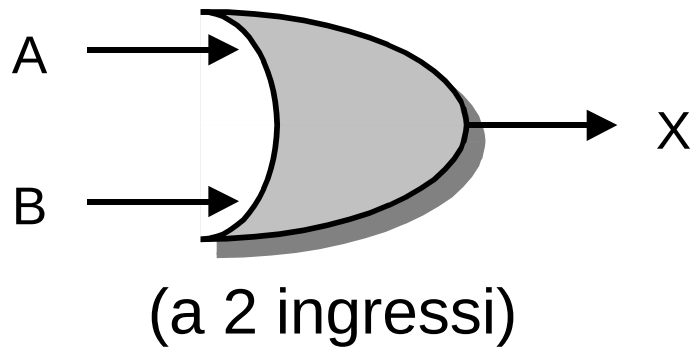


Tabella di verità

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

L'uscita vale 1 se e solo se almeno un ingresso vale 1



NAND (operatore funzionalmente completo)

Simbolo funzionale

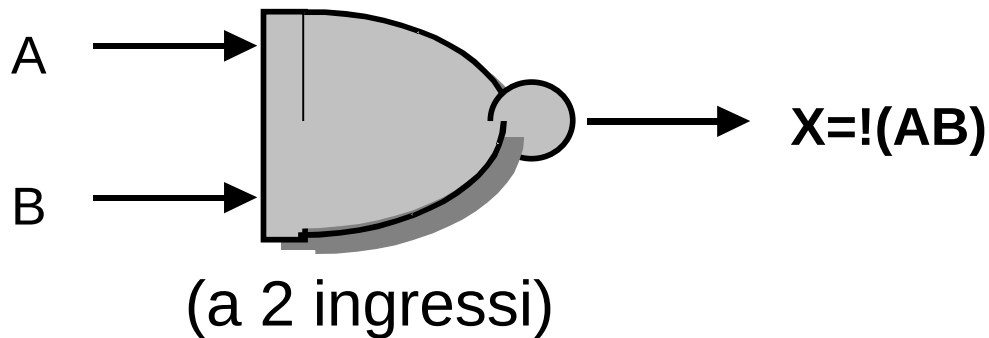


Tabella di verità

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

L'uscita vale 0 se e solo se entrambi gli ingressi valgono 1



NOR (operatore funzionalmente completo)

Simbolo funzionale

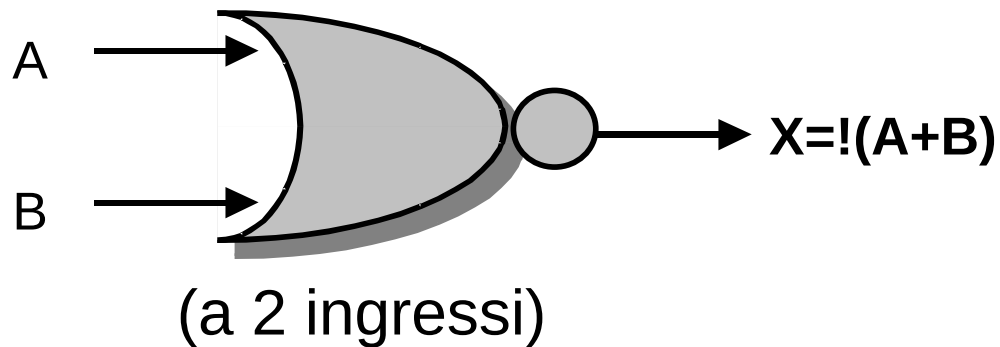


Tabella di verità

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

L'uscita vale 0 se e solo se almeno un ingresso vale 1



Altri operatori: OR esclusivo (XOR)

- XOR a 2 ingressi: l'uscita vale 1 se e solo se una sola variabile vale 1 (diseguaglianza)

Simbolo funzionale

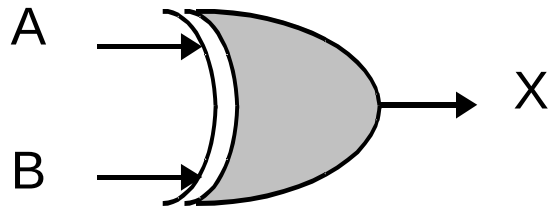


Tabella di verità

A	B	$X = !AB + A!B$
0	0	0
0	1	1
1	0	1
1	1	0



Altri operatori: XNOR

- XNOR a 2 ingressi: l'uscita vale 1 se e solo se entrambe le variabili valgono 0 o 1 (eguaglianza)

Simbolo funzionale

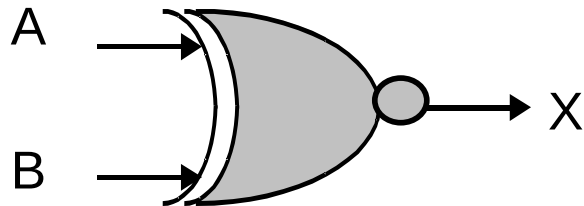


Tabella di verità

A	B	$X = AB + !A!B$
0	0	1
0	1	0
1	0	0
1	1	1



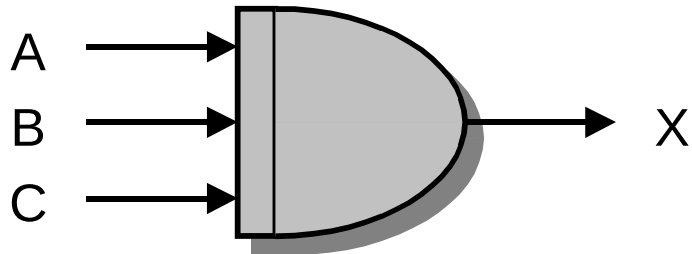
Generalizzazioni

- ❑ Alcuni tipi di porte a 2 ingressi si possono generalizzare a 3, 4, ecc. ingressi
- ❑ Le **porte a più ingressi** maggiormente usate sono la porta AND e la porta OR: tipicamente si usano AND (o OR) a 2, 4 o 8 ingressi (raramente più di 8)
- ❑ XOR generalizzato a n variabili di ingresso: l'uscita vale 1 se e solo se il numero di 1 è dispari
- ❑ XNOR generalizzato a n variabili di ingresso: l'uscita vale 1 se e solo se il numero di 1 è pari
- ❑ **Nota bene:** non tutti i tipi di **porte a più di 2 ingressi si possono realizzare come generalizzazioni di porte a 2 ingressi** (funziona sempre con AND, OR, XOR, XNOR)



Porta AND a 3 ingressi

Simbolo funzionale



L'uscita vale 1 se e solo se
tutti e 3 gli ingressi valgono 1

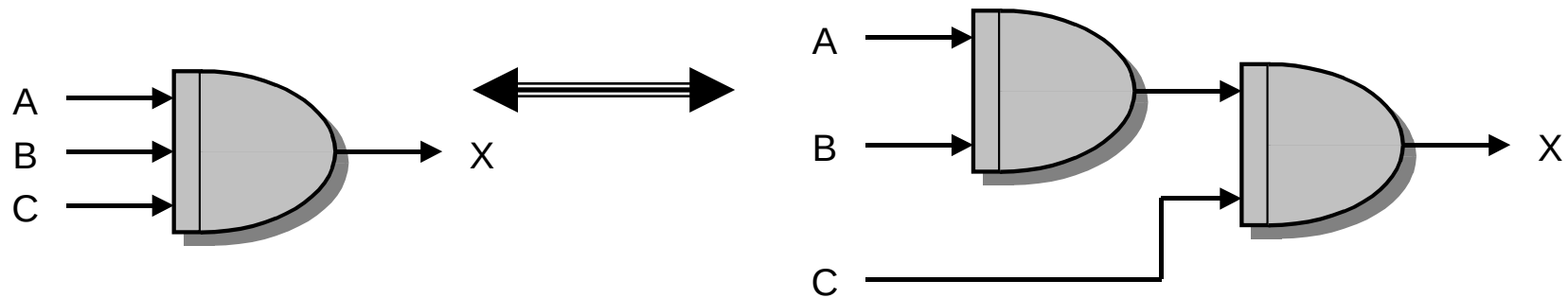
Tabella di verità

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



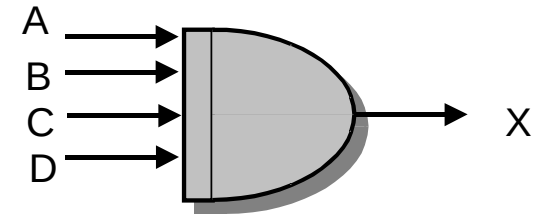
Realizzazione a cascata di AND a 3 ingressi

- La porta AND a 3 ingressi si realizza come cascata di porte AND a 2 ingressi (ma non è l'unico modo)

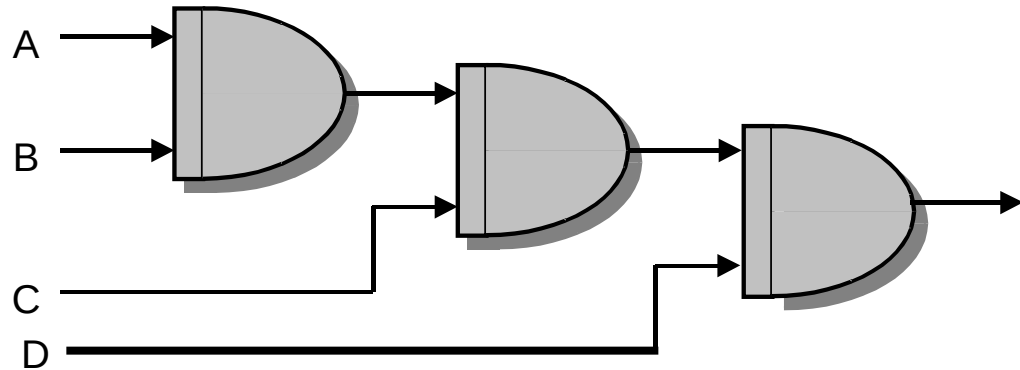




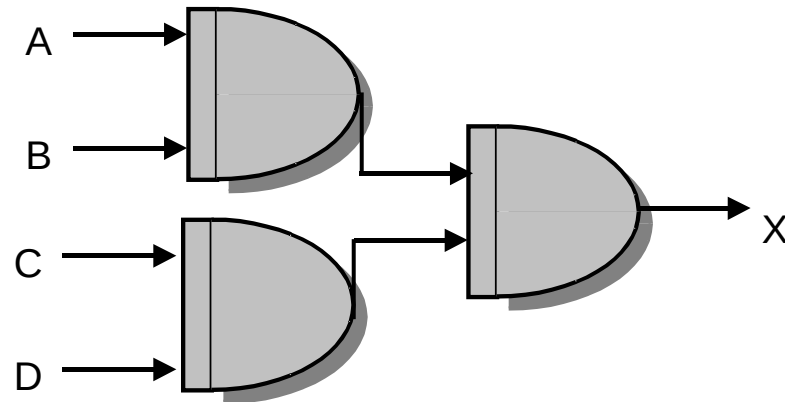
Esempio: AND a 4 ingressi



Circuito a cascata



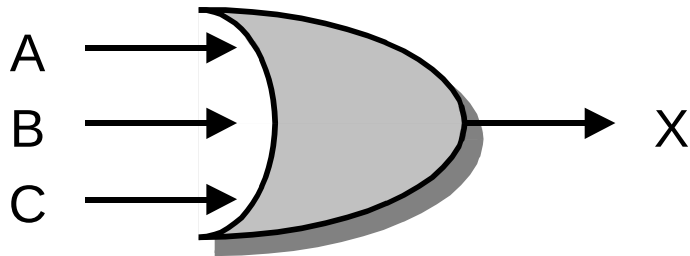
Circuito ad albero





Porta OR a 3 ingressi

Simbolo funzionale



L'uscita vale 1 se e solo se almeno un ingresso vale 1

Tabella di verità

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



Costo e velocità di una porta logica

□ Costo di realizzazione

- Il numero di transistor per realizzare una porta dipende dalla tecnologia, dalla funzione e dal numero di ingressi
- Porta NOT: 1 oppure 2 transistor, porte AND e OR: 3 oppure 4 transistor, altre porte: ≥ 4 transistor

□ Tempo di propagazione

- Il tempo di commutazione di una porta dipende dalla tecnologia, dalla funzione e dal numero di ingressi
- Le porte più veloci (oltre che più piccole) sono tipicamente le porte NAND e NOR a 2 ingressi: possono commutare in meno di 1 nanosecondo (10^{-9} sec, un miliardesimo di sec)

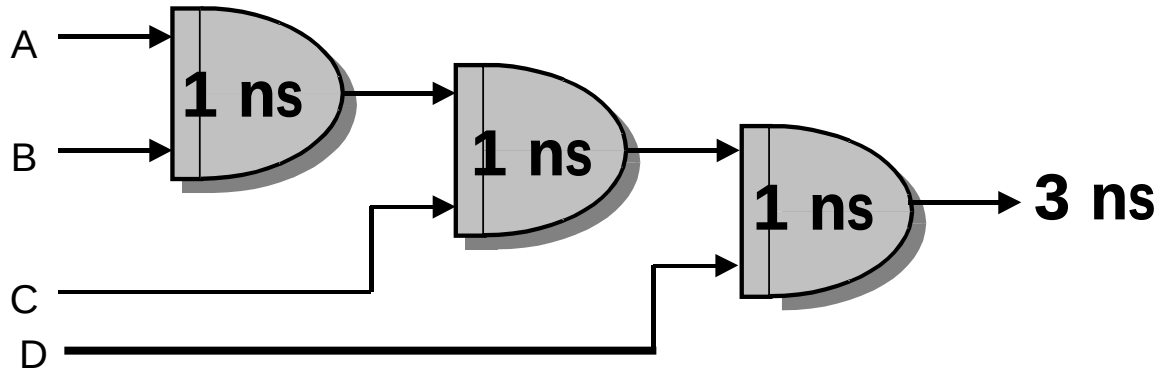
- Il costo delle porte logiche e la **velocità di commutazione** consentono di calcolare un'indicazione del costo della rete logica che realizza un'espressione booleana e del ritardo di propagazione associato alla rete stessa



Costo e tempo di propogazione (percorso critico)

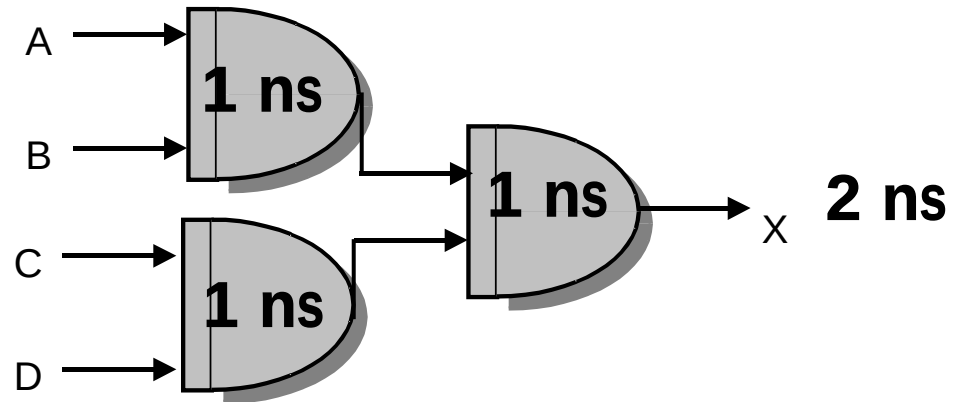
Circuito a cascata:

Costo = 3 porte AND
Ritardo totale = 3 ns
(percorso critico)



Circuito ad albero:

Costo = 3 porte AND
Ritardo totale = 2 ns





Analisi di reti combinatorie



Rete combinatoria (1)

- ❑ Ad ogni **funzione combinatoria**, data come espressione booleana, si può sempre associare un circuito digitale, formato da porte logiche, che viene chiamato **rete combinatoria**
- ❑ Gli ingressi della rete combinatoria sono le variabili della funzione
- ❑ L'uscita della rete combinatoria emette il valore assunto dalla funzione combinatoria



Rete combinatoria (2)

- Una **rete combinatoria** è un circuito digitale:
 - dotato di $n \geq 1$ ingressi principali e di un'uscita;
 - formato da porte logiche elementari AND, OR e NOT (eventualmente anche da altri tipi di porte come selettori o decodificatori);
 - privo di retroazioni.



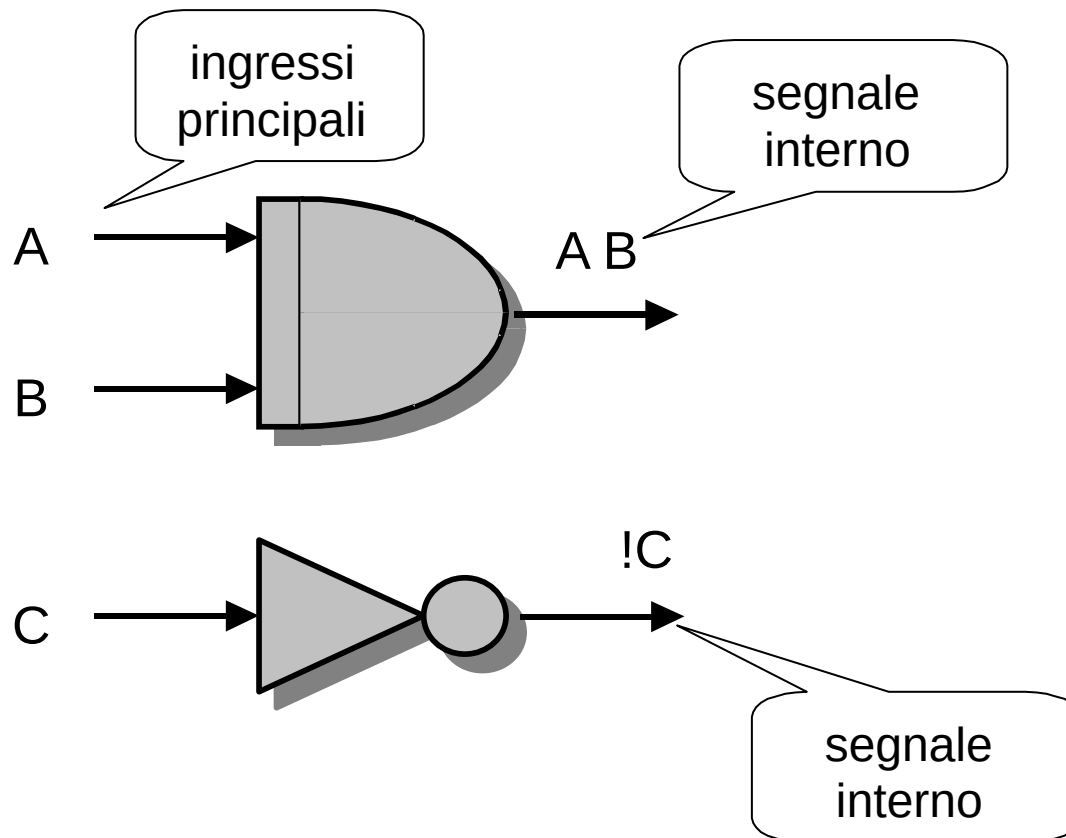
Dalla funzione logica al circuito

- A partire dalla funzione logica è possibile costruire la rete combinatoria considerando:
 - Le variabili e le variabili negate;
 - Ogni termine dell'espressione logica è sostituito dalla corrispondente porta logica fondamentale;
 - Le uscite corrispondenti ad ogni termine si compongono come indicato dagli operatori per costruire il circuito logico.



Dalla funzione logica al circuito: Esempio

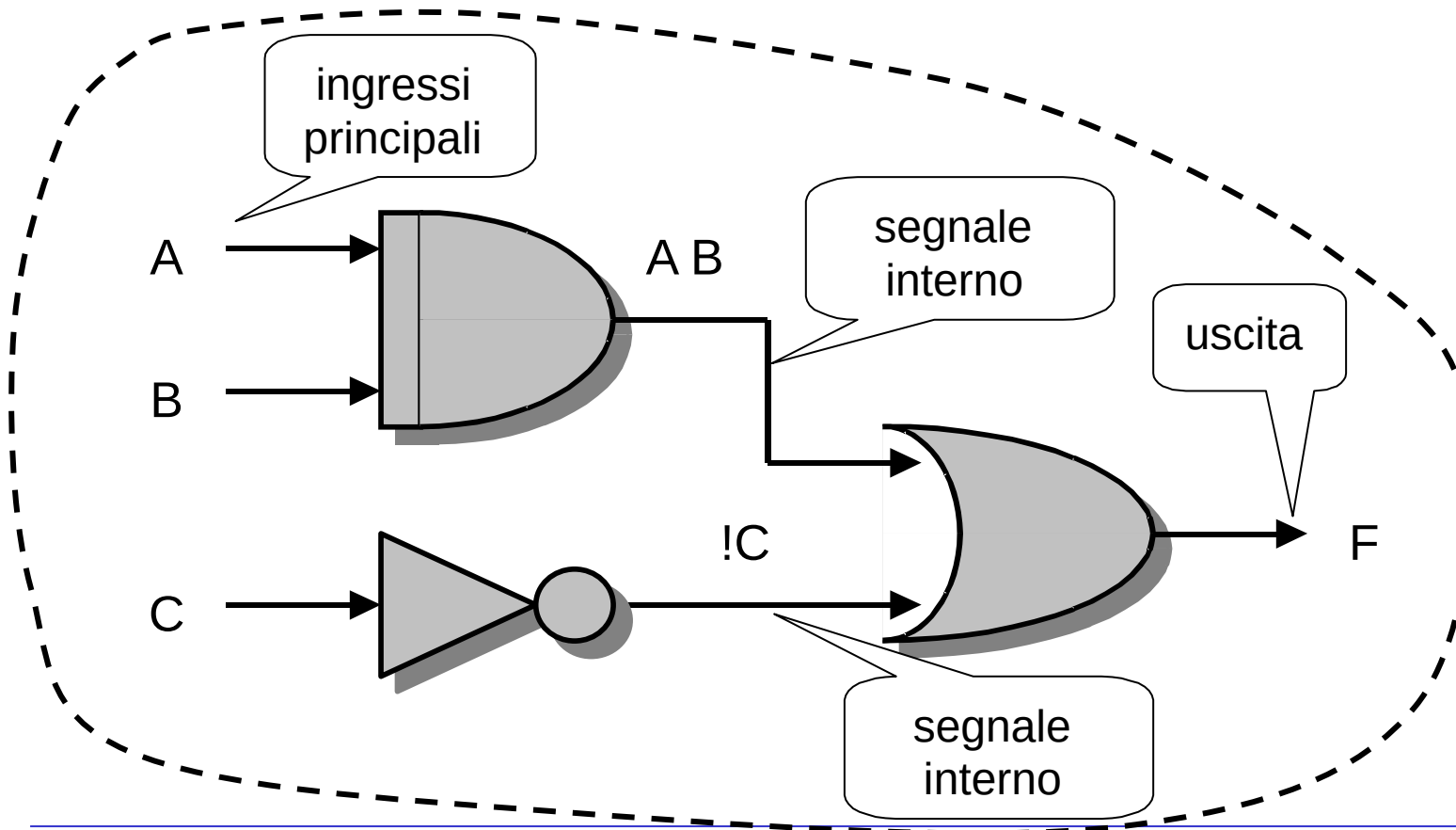
$$F(A, B, C) = A B + !C$$





Dalla funzione logica al circuito: Esempio

$$F(A, B, C) = A B + !C$$





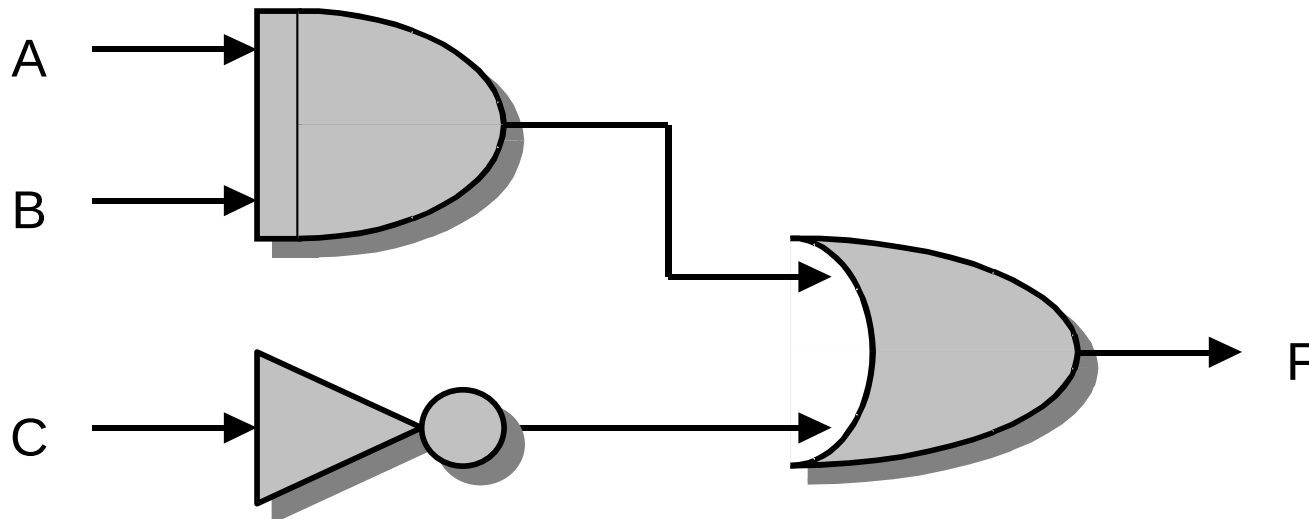
Analisi di reti combinatorie: dal circuito alla funzione logica

- In generale, dato un circuito (o rete combinatoria), è possibile ricavare la funzione logica calcolata.



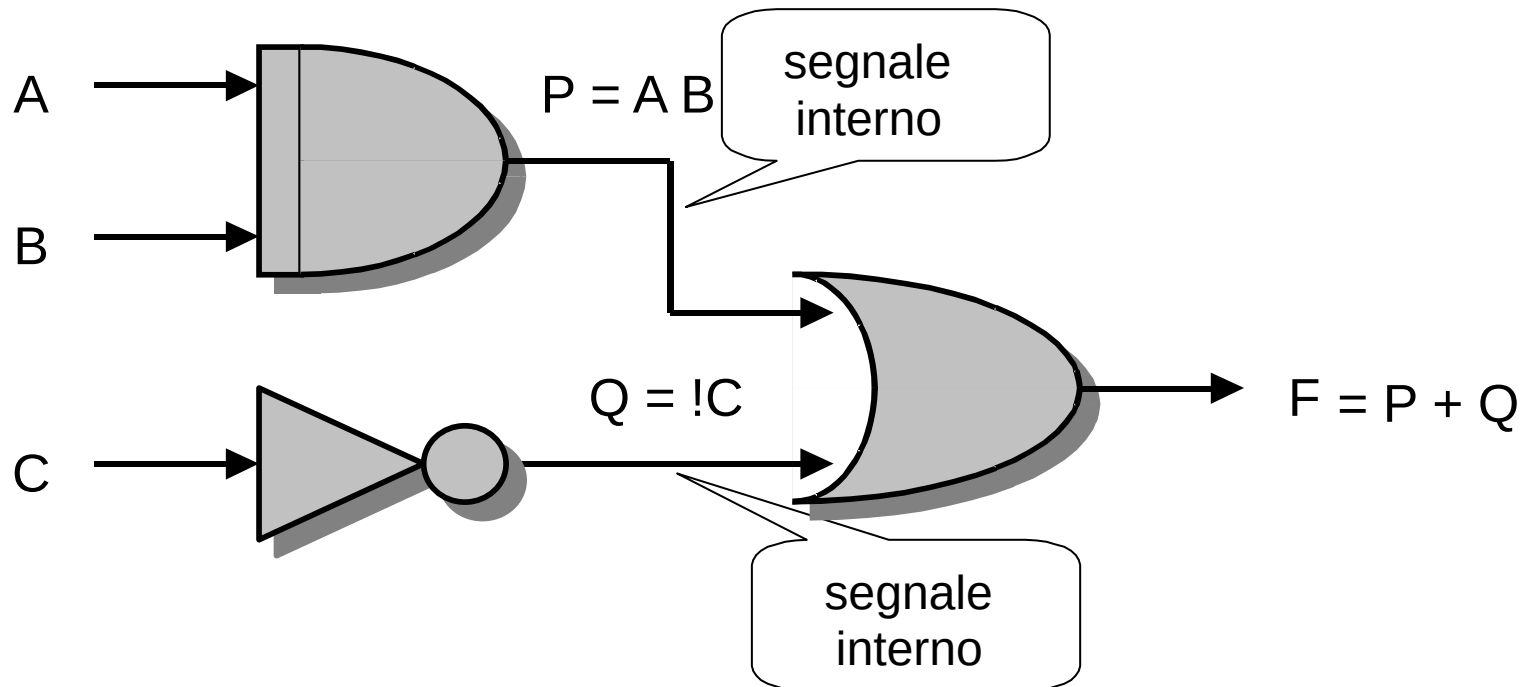
Analisi di reti combinatorie: dal circuito alla funzione logica: Esempio 1

schema o circuito logico



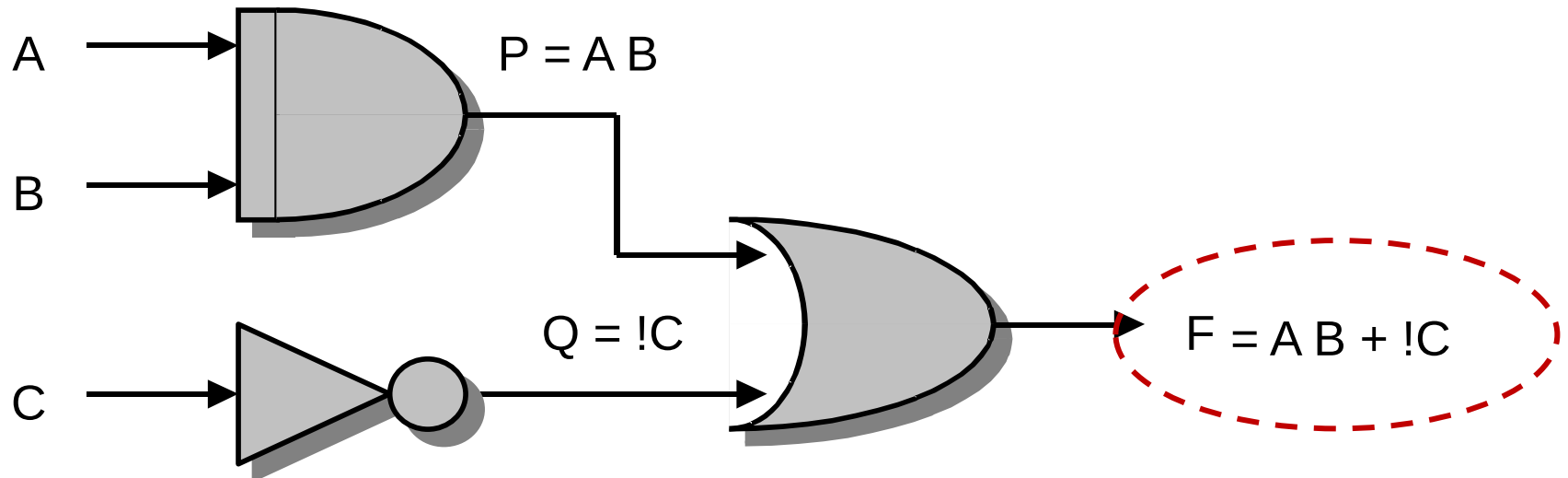


Analisi di reti combinatorie: dal circuito alla funzione logica: Esempio 1





Analisi di reti combinatorie: dal circuito alla funzione logica: Esempio 1





Analisi di reti combinatorie: dal circuito alla funzione logica: Esempio 1

Tabella di verità
(per comodità è riportato
anche il calcolo)

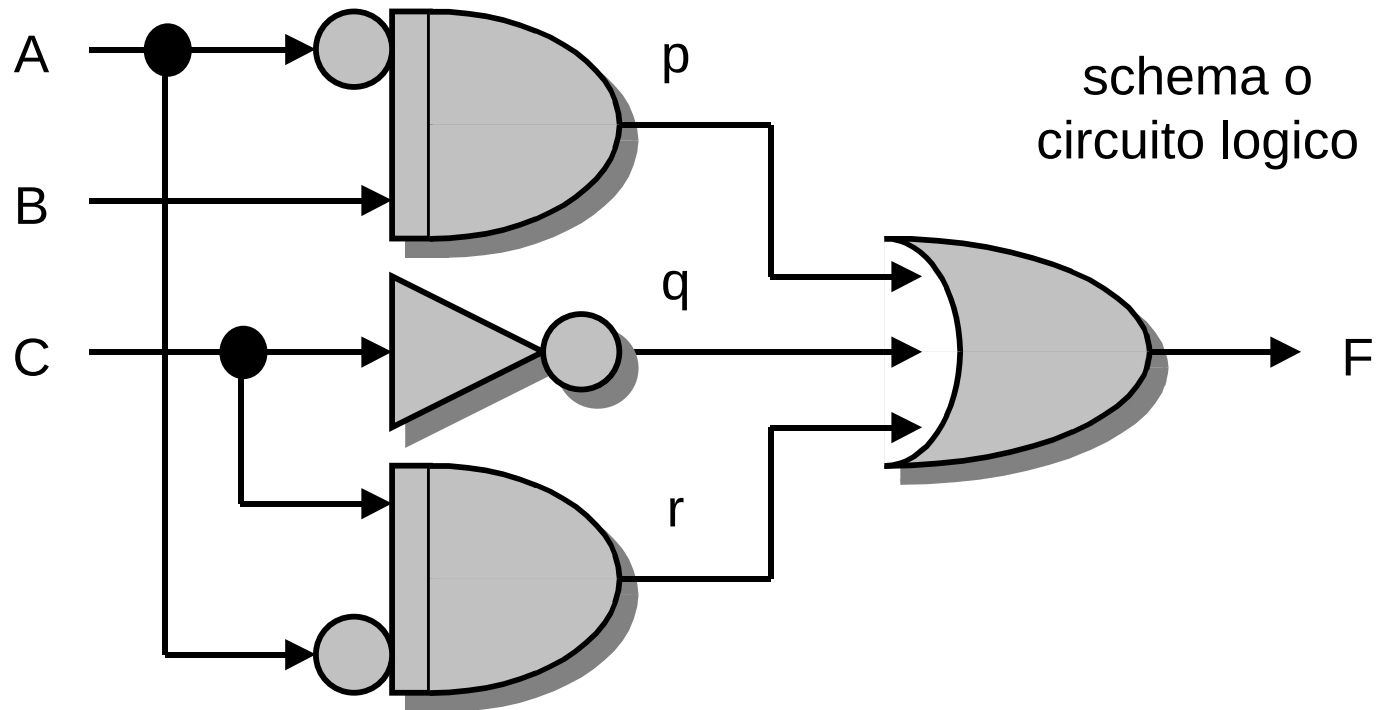
# riga	A	B	C	$A B + !C$	F
0	0	0	0	$0 0 + !0$	1
1	0	0	1	$0 0 + !1$	0
2	0	1	0	$0 1 + !0$	1
3	0	1	1	$0 1 + !1$	0
4	1	0	0	$1 0 + !0$	1
5	1	0	1	$1 0 + !1$	0
6	1	1	0	$1 1 + !0$	1
7	1	1	1	$1 1 + !1$	1

$$F(A, B, C) = A B + !C$$



Analisi di reti combinatorie: dal circuito alla funzione logica – Esempio 2

(1) Si applicano nomi ai segnali interni: p, q e r

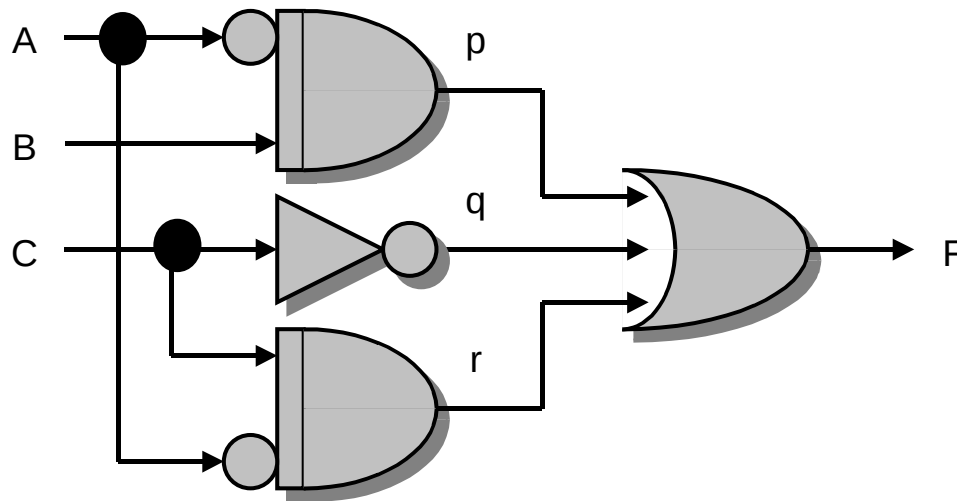




Analisi di reti combinatorie: dal circuito alla funzione logica – Esempio 2

(2) Si ricavano le espressioni booleane corrispondenti ai segnali interni:

- $p = !A B$
- $q = !C$
- $r = !A C$





Analisi di reti combinatorie: dal circuito alla funzione logica – Esempio 2

(3) Si ricava l'uscita come espressione booleana in funzione dei segnali interni:

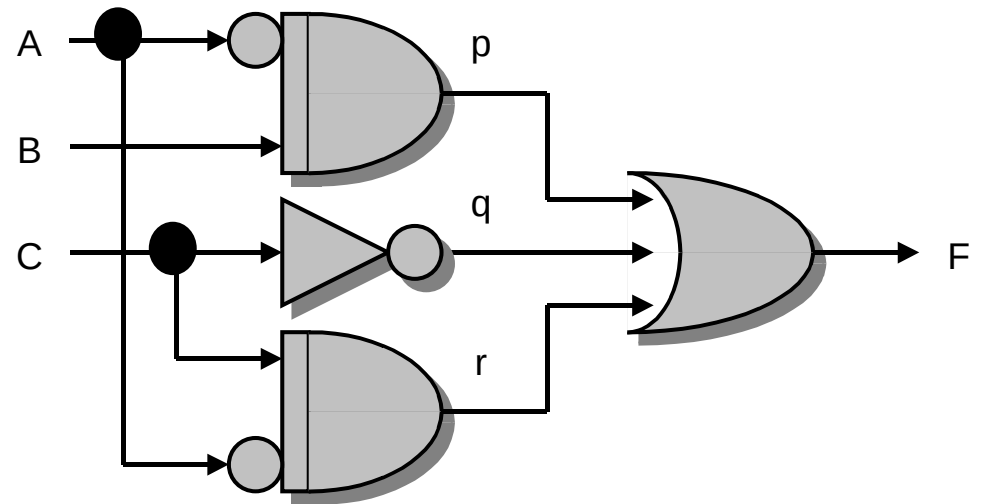
- $F = p + q + r$

(4) Per sostituzione, si ricava l'uscita come espressione booleana in funzione degli ingressi principali:

- $F = p + q + r$

- $F(A, B, C) = !A B + !C + !A C$

L'espressione booleana così trovata ha una struttura conforme allo schema logico di partenza





Analisi di reti combinatorie: dal circuito alla funzione logica – Esempio 2

Tabella di verità
(per comodità è riportato
anche il calcolo)

# riga	A	B	C	$\neg A B + \neg C + \neg A C$	F
0	0	0	0	$\neg 0 0 + \neg 0 + \neg 0 0$	1
1	0	0	1	$\neg 0 0 + \neg 1 + \neg 0 1$	1
2	0	1	0	$\neg 0 1 + \neg 0 + \neg 0 0$	1
3	0	1	1	$\neg 0 1 + \neg 1 + \neg 0 1$	1
4	1	0	0	$\neg 1 0 + \neg 0 + \neg 1 0$	1
5	1	0	1	$\neg 1 0 + \neg 1 + \neg 1 1$	0
6	1	1	0	$\neg 1 1 + \neg 0 + \neg 1 0$	1
7	1	1	1	$\neg 1 1 + \neg 1 + \neg 1 1$	0



Simulazione circuitale

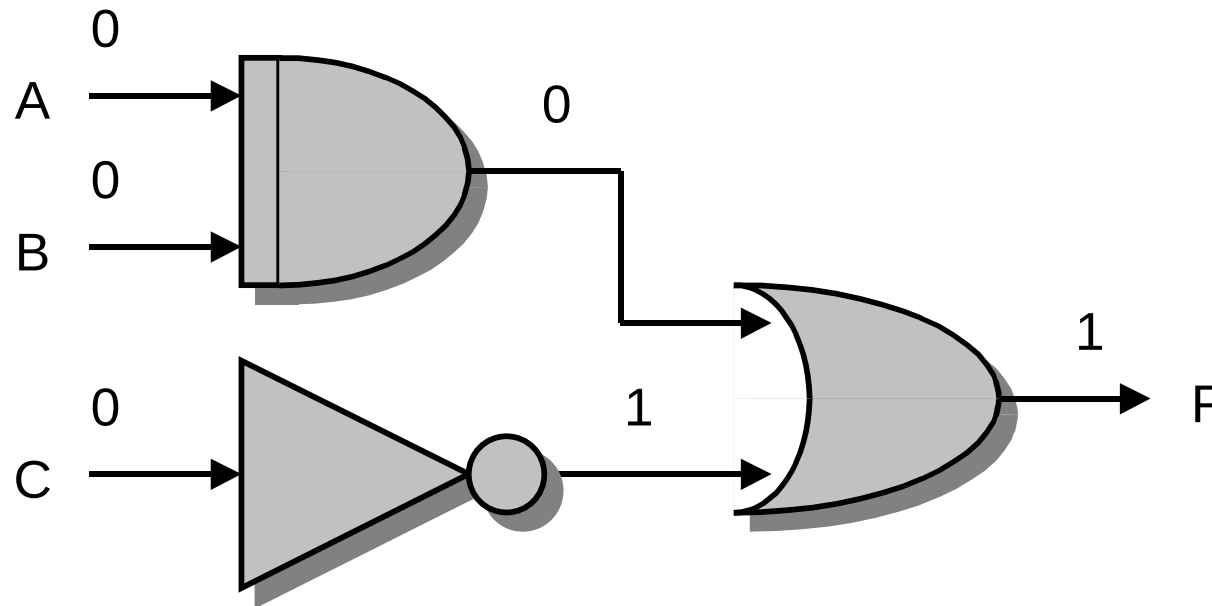
(dalla rete combinatoria alla tabella di verità)

- ❑ La tabella di verità di una rete combinatoria può anche essere ricavata per **simulazione** del funzionamento circuitale della rete combinatoria stessa
- ❑ Per simulare il funzionamento circuitale di una rete combinatoria, si applicano tutte le possibili combinazioni dei valori agli ingressi, e li si propaga lungo la rete fino all'uscita
- ❑ Ad esempio per una rete a **3** ingressi bisogna svolgere **$2^3 = 8$** simulazioni circuitali



Simulazione circuitale: Esempio 1

(corrisponde alla riga 0 della tabella)

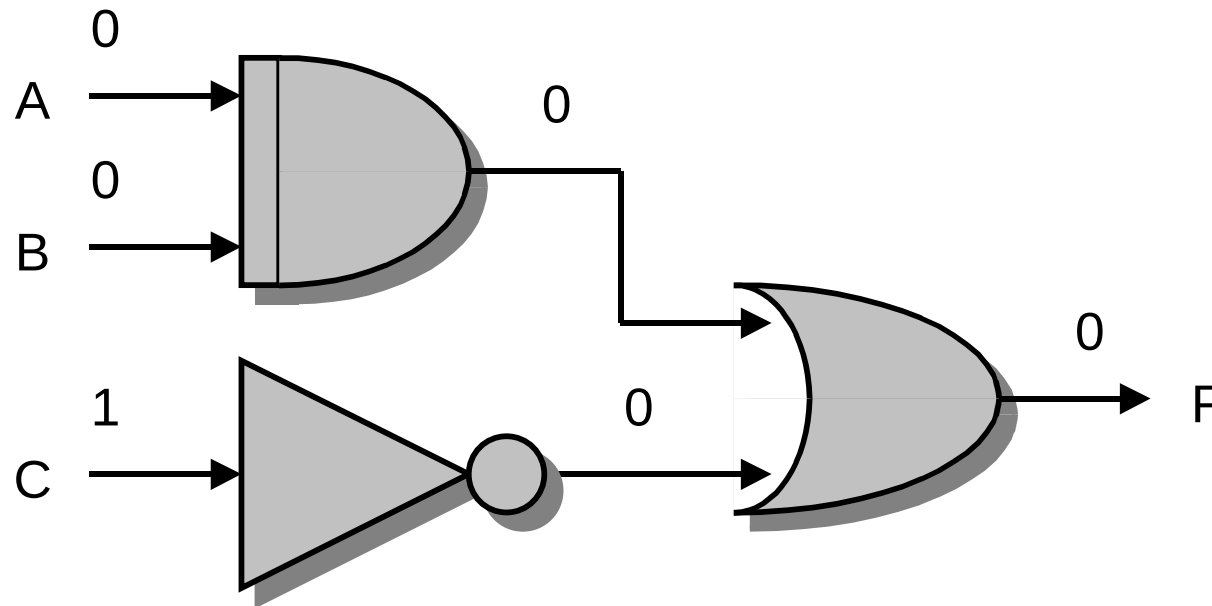


Risultato della simulazione: $F(0, 0, 0) = 1$



Simulazione circuitale: Esempio 1

(corrisponde alla riga 1 della tabella)

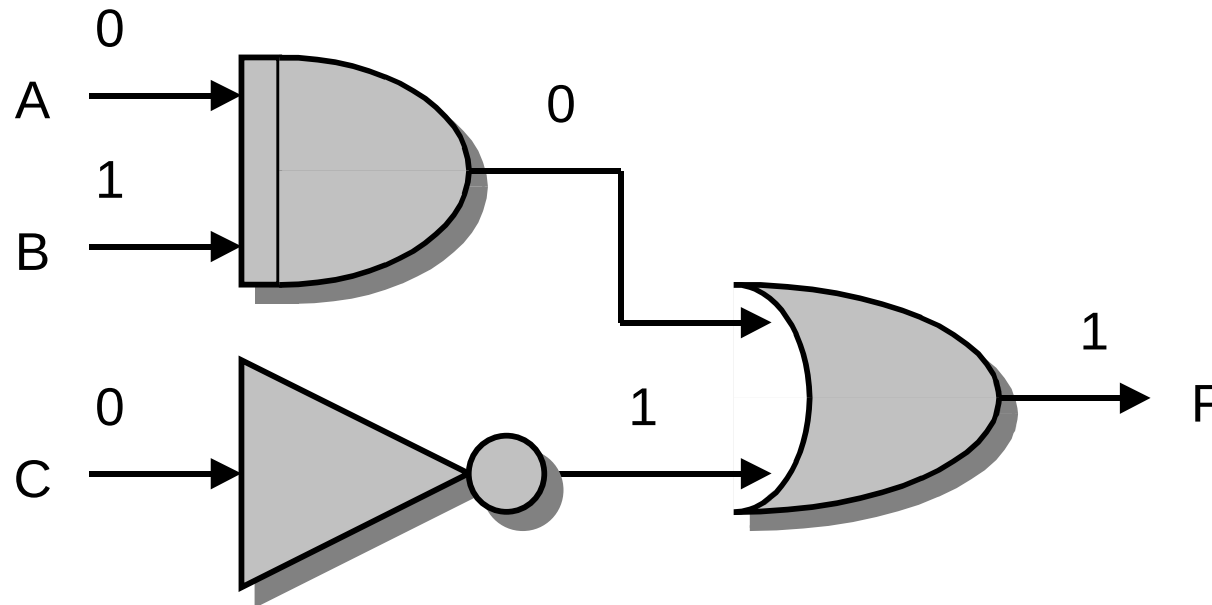


Risultato della simulazione: $F(0, 0, 1) = 0$



Simulazione circuitale: Esempio 1

(corrisponde alla riga 2 della tabella)

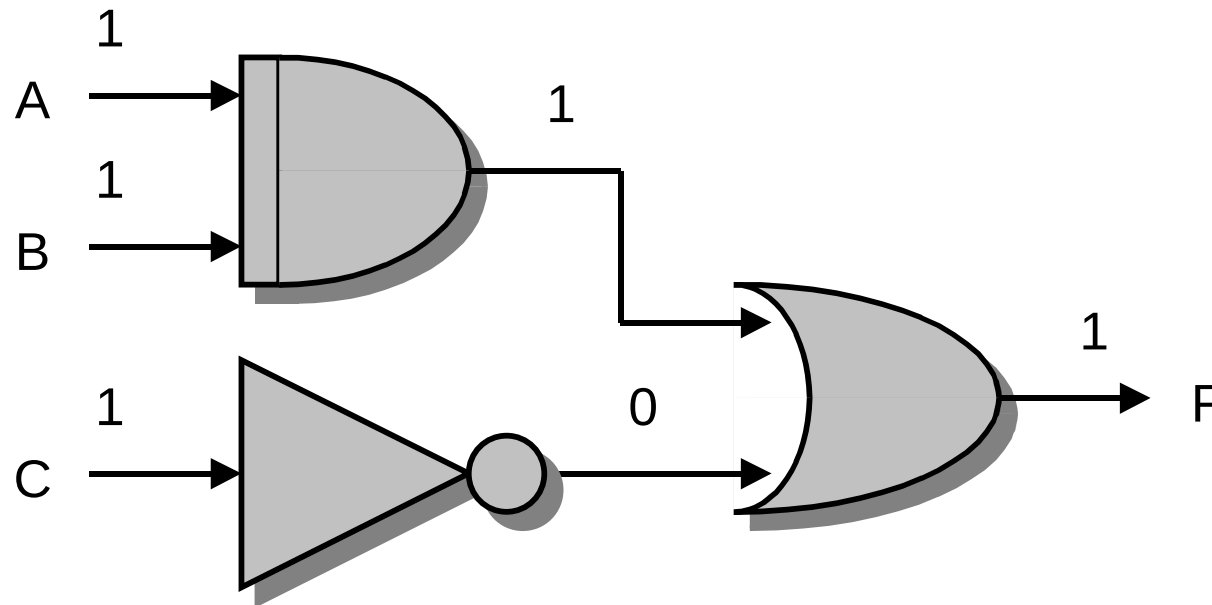


Risultato della simulazione: $F(0, 1, 0) = 1$



Simulazione circuitale: Esempio 1

.....
(corrisponde alla riga 7 della tabella)



Risultato della simulazione: $F(1, 1, 1) = 1$



Risultato delle simulazioni circuitali: Esempio 1

Tabella di verità

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

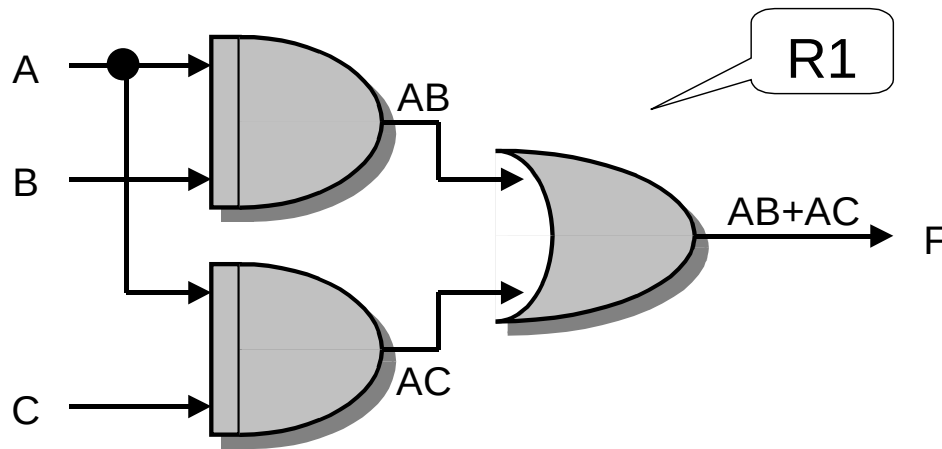


Reti combinatorie equivalenti

- ❑ Una funzione combinatoria può ammettere **più reti combinatorie differenti** che la sintetizzano
- ❑ Reti combinatorie che realizzano la medesima funzione combinatoria si dicono **equivalenti**
- ❑ Esse hanno tutte la stessa funzione (tabella delle verità), ma possono avere struttura (e costo) differente



Due reti equivalenti



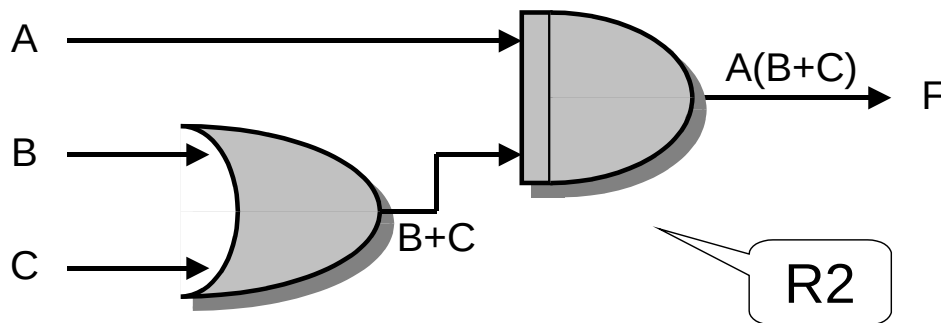
$$\mathbf{F1} = AB + AC$$

Trasformazione:

$$F1 = AB + AC =$$

$$= A(B + C) = F2$$

(prop. distributiva)



$$\mathbf{F2} = A(B + C)$$



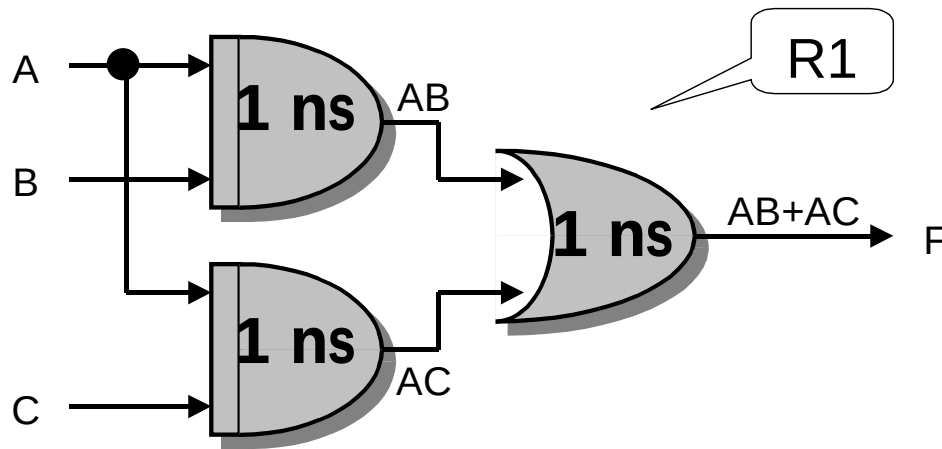
Costo e velocità di reti combinatorie

- Il **costo di una rete combinatoria** si valuta in vari modi (criteri di costo):
 - Numero di porte, per tipo di porta e per quantità di ingressi della porta
 - Numero di porte universali (NAND o NOR)
 - Numero di transistor
 - Complessità delle interconnessioni
 - e altri ancora ...

- La **velocità di una rete** combinatoria è misurata dal tempo che una variazione di ingresso impiega per modificare l'uscita della rete (o **ritardo di propagazione**)
 - Per calcolare la velocità di una rete combinatoria, occorre conoscere i ritardi di propagazione delle porte logiche componenti la rete, e poi analizzare i **percorsi ingressi-uscita**



Due reti equivalenti: Costo e tempo di propag.

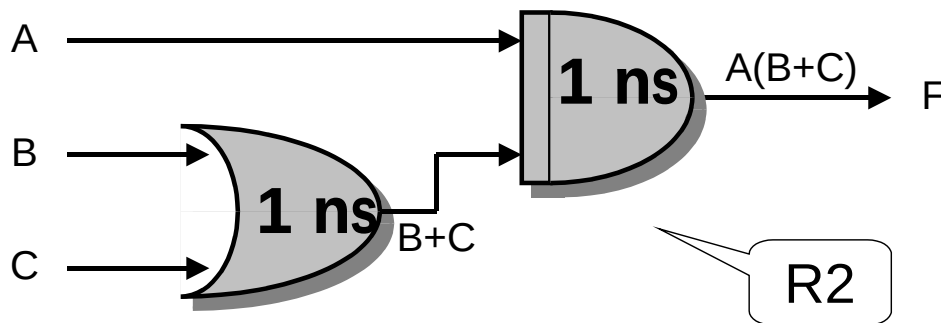


$$\mathbf{F1} = AB + AC$$

Costo = 3 porte elem.

Ritardo totale = 2 ns

Frequenza = $1 / 2 \text{ ns} = 500 \text{ MHz}$



$$\mathbf{F2} = A(B + C)$$

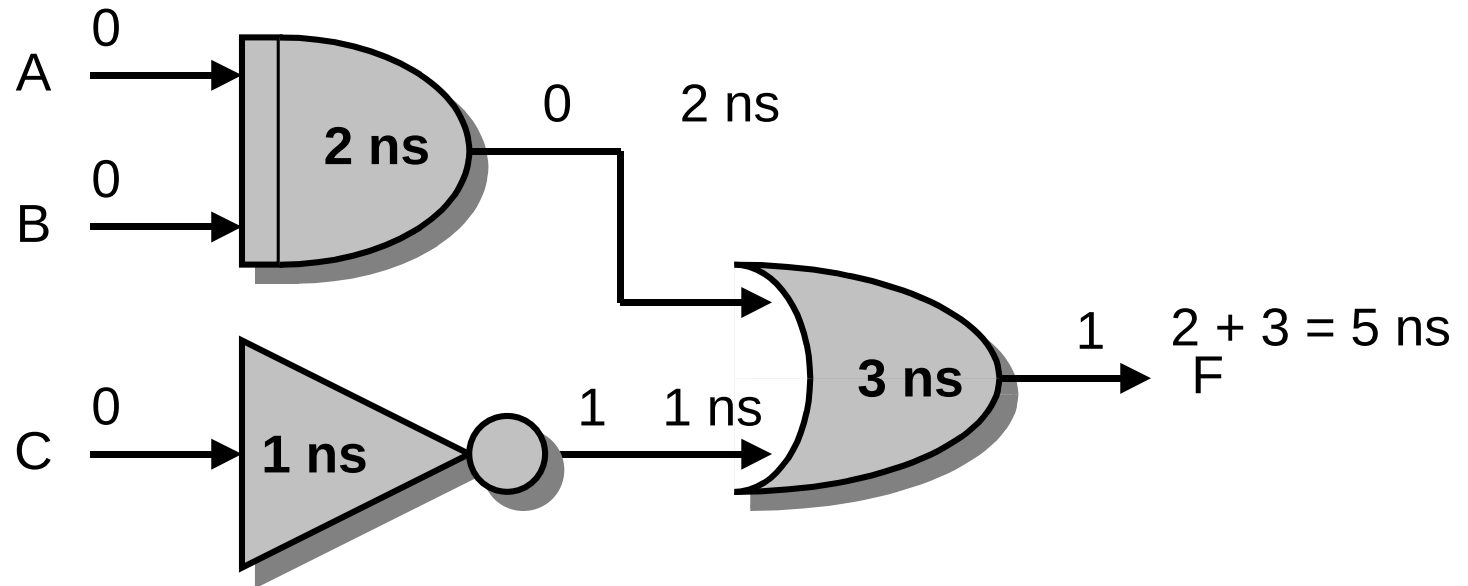
Costo = 2 porte elem.

Ritardo totale = 2 ns (percorso critico)

Frequenza = $1 / 2 \text{ ns} = 500 \text{ MHz}$



Tempo di propagazione: percorso critico



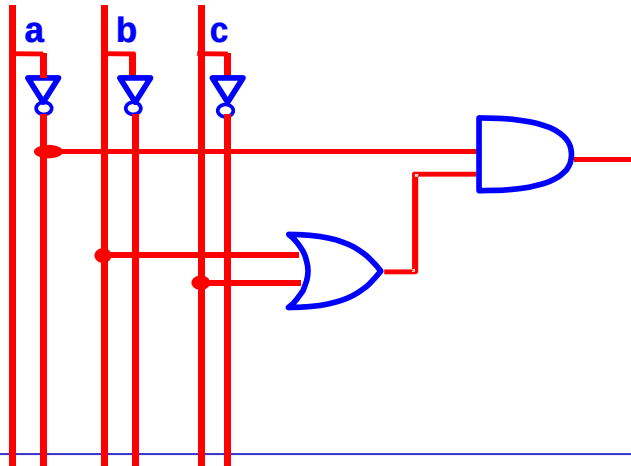
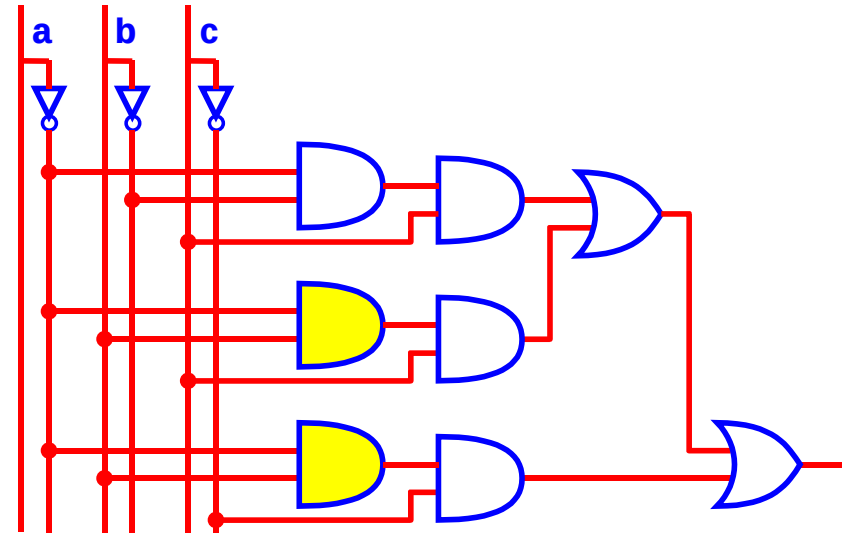
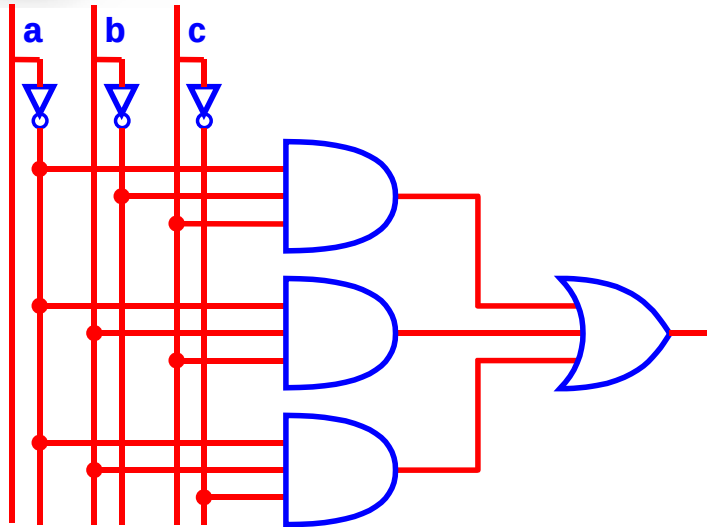
Ritardo totale = 5 ns = $5 \cdot 10^{-9}$ sec (percorso critico)

Freq. di commutazione = $1 / 5 \text{ ns} = 200 \text{ MHz}$



Costo e ritardo di propagazione di reti equivalenti (porte a 2 ingressi)

$$F(a,b,c)=!a!bc + !abc + !ab!c = !a(c+b)$$



Porte a 2 ingressi

AND: Costo = 4, Rit. = 10

OR: Costo = 4, Rit. = 12

NOT: Costo = 1, Rit. = 2



Sintesi di reti combinatorie



Sintesi di reti combinatorie

- ❑ La sintesi di una rete combinatoria espressa come tabella delle verità, consiste nel ricavare lo schema logico (il circuito digitale) che calcola la funzione combinatoria
- ❑ In generale, per una data **tabella di verità** possono esistere **più reti combinatorie** (la soluzione al problema di sintesi non è dunque unica)
- ❑ Esistono svariate procedure di sintesi di reti combinatorie, che differiscono per:
 - Complessità della procedura di sintesi
 - Ottimalità della rete combinatoria risultante, per dimensioni e velocità



Sintesi di reti combinatorie: forme canoniche

- Data una **funzione booleana**, la soluzione iniziale al problema di determinare una sua espressione consiste nel ricorso alle **forme canoniche**.
- Le forme canoniche sono due:
 - **Somma di prodotti (SoP)** (sintesi in 1a forma canonica)
 - **Prodotto di somme (PoS)** (sintesi in 2a forma canonica).
- Data una funzione booleana esistono **una ed una sola** forma canonica SoP ed una e una sola forma PoS che la rappresenta.



Sintesi in 1^a forma canonica (o sintesi come somma di prodotti: SoP)

Data una tabella di verità, a $n \geq 1$ ingressi, della funzione da sintetizzare, la **funzione F** che la realizza può essere **specificata** come

- la **somma logica (OR)** di tutti (e soli) i **termini prodotto (AND)** delle **variabili di ingresso corrispondenti agli 1** della funzione.



1^a forma canonica

- Si consideri il seguente esempio:

a	b	$f(a, b)$
0	0	0
0	1	1
1	0	0
1	1	1

- È intuitivo osservare che la funzione possa essere ottenuta dal OR delle seguenti funzioni:

a	b	$f(a, b)$	=	a	b	$f_1(a, b)$	+	a	b	$f_2(a, b)$
0	0	0		0	0	0		0	0	0
0	1	1		0	1	1		0	1	0
1	0	0		1	0	0		1	0	0
1	1	1		1	1	0		1	1	1



1^a forma canonica

- Per cui, intuitivamente, si ottiene:

a	b	$f(a, b)$
0	0	0
0	1	1
1	0	0
1	1	1

=

a	b	$f_1(a, b)$
0	0	0
0	1	1
1	0	0
1	1	0

+

a	b	$f_2(a, b)$
0	0	0
0	1	0
1	0	0
1	1	1

$m_1(a, b) = \neg a \wedge b \quad m_3(a, b) = ab$

- Quando $a=0$ e $b=1$ il prodotto $\neg a \wedge b$ assume valore 1 mentre vale 0 in tutti gli altri casi.
- Quando $a=1$ e $b=1$ il prodotto ab assume valore 1 mentre vale 0 in tutti gli altri casi.



1^a forma canonica

- Ne consegue:

a	b	f(a, b)	=	a	b	f ₁ (a, b)	+	a	b	f ₂ (a, b)
0	0	0		0	0	0		0	0	0
0	1	1		0	1	1		0	1	0
1	0	0		1	0	0		1	0	0
1	1	1		1	1	0		1	1	1



$$f(a, b) = \neg a \, b + ab$$

- Mettendo in OR i **mintermini** della funzione si ottiene l'espressione *booleana* della funzione stessa espressa come somma di prodotti.



Mintermine

- ❑ Un **mintermine** m_i è una funzione booleana a n ingressi che vale **1** in corrispondenza della sola configurazione di ingresso che vale i .
 - ❑ Esempio: $m_1(a,b) = !ab$
 - ❑ Un **mintermine** m_i corrisponde al **prodotto** di n letterali, ciascuno dei quali compare nella forma x se nella corrispondente configurazione di ingresso ha valore **1**, nella forma $!x$ se ha valore **0**.
 - ❑ Data una funzione ad n ingressi, possiamo avere fino a 2^n mintermini.
 - ❑ Ogni **mintermine** è rappresentabile con un **AND** a n ingressi.
-



Esempio: Funzione di maggioranza

- Si chiede di sintetizzare (in 1^a forma canonica) una funzione combinatoria dotata di 3 ingressi A, B e C, e di un'uscita F, funzionante come segue:
 - Se la maggioranza degli ingressi vale 0, l'uscita vale 0
 - Se la maggioranza degli ingressi vale 1, l'uscita vale 1



Esempio: Tabella di verità

- L'uscita vale 1 se e solo se 2 o tutti e 3 gli ingressi valgono 1 (cioè se e solo se il valore 1 è in maggioranza)

mintermini

# riga	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1



Esempio: Espressione booleana

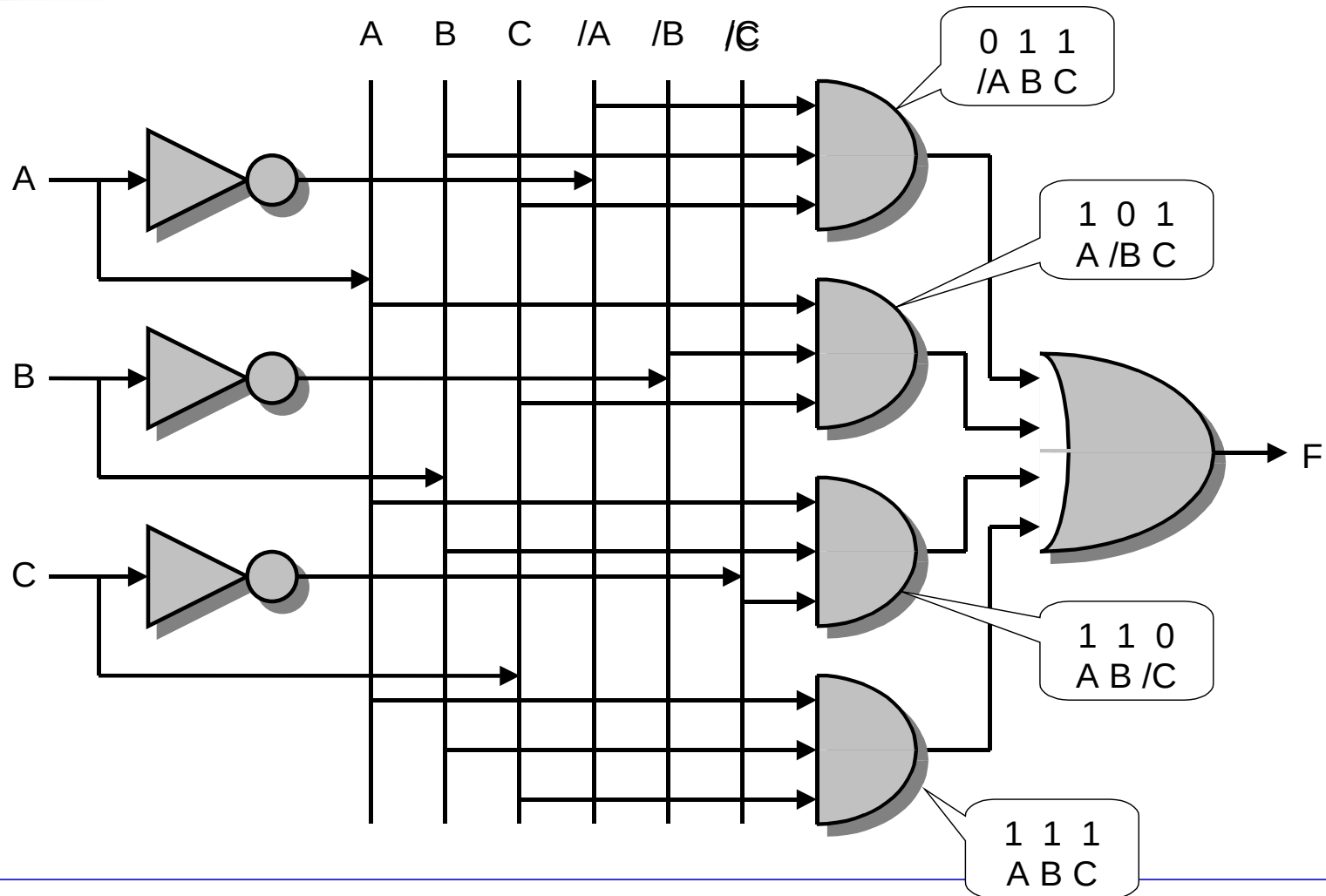
- Dalla **tabella di verità** si ricava, tramite la sintesi in 1a forma canonica (somma di prodotti), **l'espressione booleana** che rappresenta la funzione

$$F(A, B, C) = !A B C + A !B C + A B !C + A B C$$

- Dall'**espressione booleana** si ricava la **rete combinatoria** (il circuito) che è sempre **a 2 livelli (Somma di Prodotti)**



Esempio: Rete combinatoria SoP





Esempio: Minimizzazione

$$F(A, B, C) = !A B C + A !B C + A B !C + A B C$$

Espressione trasformata	Proprietà utilizzata
$!A B C + A !B C + A B !C + A B C$	Idempotenza $X + X = X$
$!A B C + A B C + A !B C + A B C + A B !C + A B C$	Distributiva $XY + XZ = X(Y+Z)$
$B C (!A + A) + A C (!B + B) + A B (!C + C)$	Inverso $(!X + X) = 1$
$BC 1 + AC 1 + AB 1$	Identità $X 1 = X$
$BC + AC + AB$	



Sintesi in 2^a forma canonica (o sintesi come prodotto di somme: PoS)

Data una tabella delle verità, a $n \geq 1$ ingressi, della funzione da sintetizzare, la **funzione F** che la realizza può essere **specificata** come

- il **prodotto logico (AND)** di tutti (e soli) i **termini somma (OR)** delle **variabili di ingresso corrispondenti agli 0** della funzione.
- Ogni **termine somma** (o **maxtermine**) è costituito dalla somma logica delle variabili di ingresso (letterale) prese in forma naturale se valgono 0, in forma complementata se valgono 1.



2^a forma canonica

- Si consideri nuovamente lo stesso esempio:

a	b	$f(a, b)$
0	0	0
0	1	1
1	0	0
1	1	1

- È intuitivo osservare che la funzione possa essere ottenuta dall'AND delle seguenti funzioni:

a	b	$f(a, b)$		a	b	$f_1(a, b)$		a	b	$f_2(a, b)$
0	0	0		0	0	0		0	0	1
0	1	1		0	1	1		0	1	1
1	0	0		1	0	1		1	0	0
1	1	1		1	1	1		1	1	1

=

*



2^a forma canonica

- Per cui, intuitivamente, si ottiene:

a	b	f(a, b)	=	a	b	f ₁ (a, b)	*	a	b	f ₂ (a, b)
0	0	0		0	0	0		0	0	1
0	1	1		0	1	1		0	1	1
1	0	0		1	0	1		1	0	0
1	1	1		1	1	1		1	1	1

$$M_0(a, b) = a + b$$

$$M_2(a, b) = !a + b$$

Infatti, ad es., quando $a=0$ e $b=0$ allora $(a+b)$ assume valore 0 mentre vale 1 in tutti gli altri casi.

$$\Rightarrow f(a, b) = (a+b) * (!a + b)$$

Mettendo in AND i **maxtermini** della funzione si ottiene l'espressione *booleana* della funzione stessa espressa come prodotto di somme.

nel **maxtermine** (somma) una variabile compare nella forma naturale (x) se nella corrispondente configurazione di ingresso ha valore 0, nella forma complementata (x') se ha valore 1



Sintesi PoS

□ $F = (A+B+C)(A+B+\neg C)(A+\neg B+C)(\neg A+B+C)$

maxtermini

# riga	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1



Costo di una rete logica

- Sintesi in 1^a forma canonica: la funzione logica ottenuta è costituita da
 - n° di mintermini pari agli 1 della funzione
 - per ogni mintermine, n° di letterali pari al numero delle variabili di ingresso
 - (dualmente per la 2^a forma canonica)

- Il costo della rete è funzione sia del n° di mintermini che del numero di letterali (variabili o variabili negate).



Riassumendo...

- ❑ Ora sappiamo far corrispondere funzioni booleane combinatorie e reti combinatorie (versioni SoP e PoS)
- ❑ (sintesi) Ora sappiamo progettare qualunque rete combinatoria dato il suo comportamento ai morsetti (la tabella delle verità) usando porte logiche elementari
- ❑ (analisi) Ora sappiamo ricavare la tabella di verità dalla struttura della rete combinatoria
- ❑ Ma in questo corso non ci interessano i criteri di ottimizzazione del costo della rete e delle velocità dei suoi transitori ...